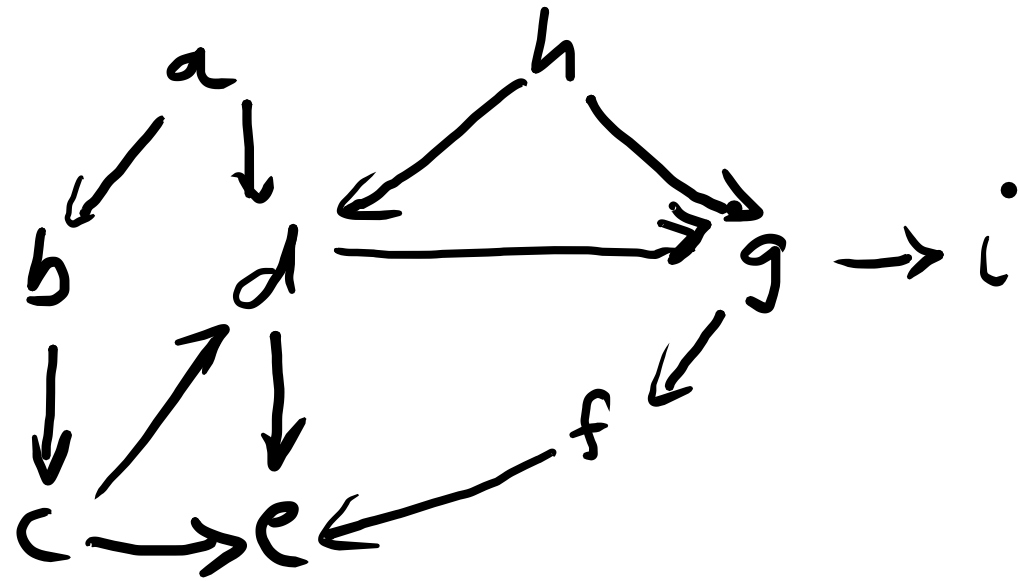


Topological order/sort: Application of DFS

Let $G(V, E)$ be a directed acyclic graph (DAG)

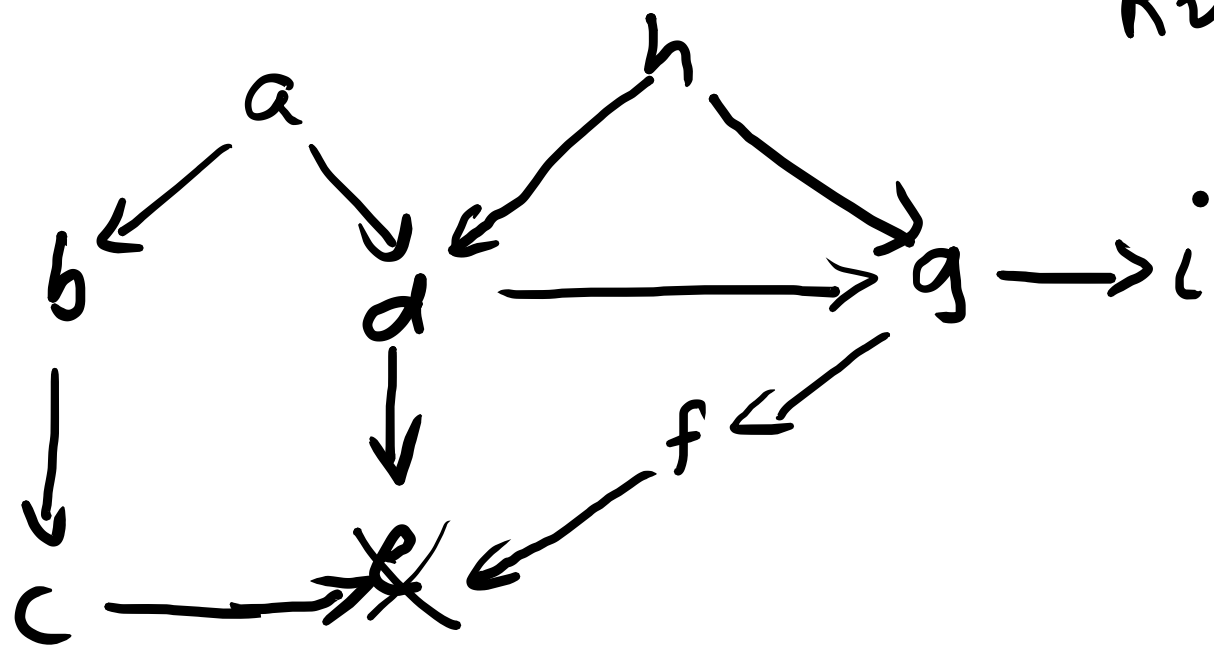
A topological order of G is an ordering of the vertices V such that for any edge (u, v) , it must hold that u precedes v in that order

$$\begin{array}{ccc} u & \xrightarrow{\quad} & v \\ & \vdots & \\ u & \leftarrow & v \end{array}$$

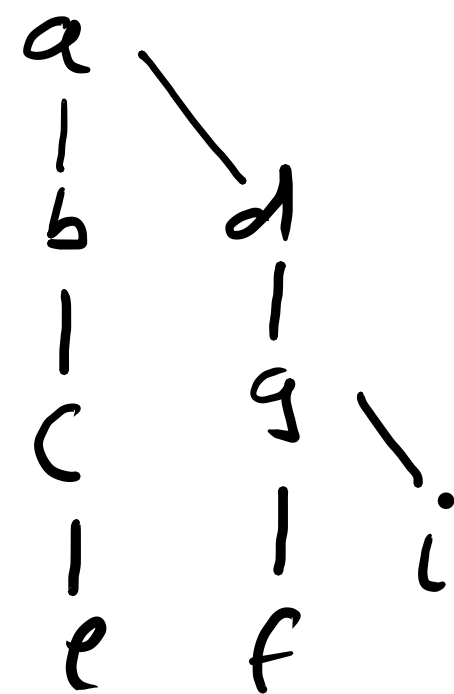


Algorithm

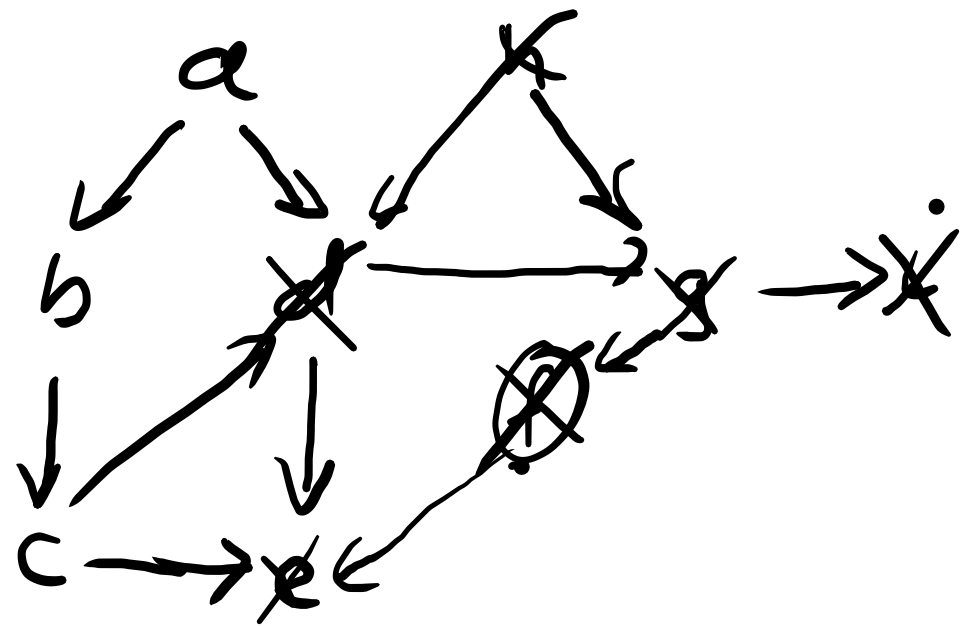
- Create an empty list L^R
 - Run DFS on G . Whenever vertex v turns BLACK (i.e. popped from stack) append it to L^R
 - Output the reverse order of L^R i.e. L
- Run DFS from vertex a and then from h



$$L^R = \{e, c, b, f, i, g, d, a, h\}$$



$$L = \{h, a, d, g, i, f, b, c, e\}$$



Time Complexity

$$L^R = \{e, f, i, g, d, h, c, b, a\}$$

$$L = \{a, b, c, h, d, g, i, f, e\}$$

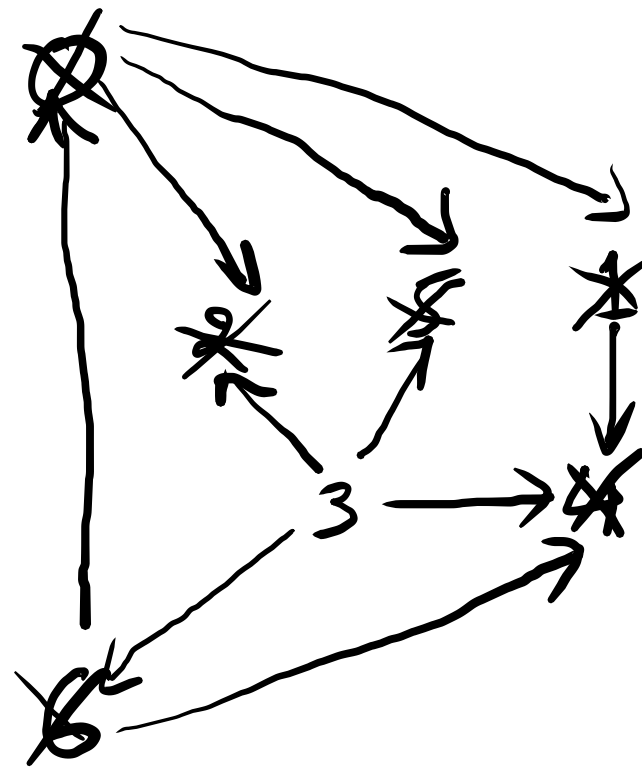
Step 1: Empty List $O(1)$

Step 2: Run DFS $O(|V| + |E|)$

Step 3: Reverse List $O(|V|)$

$$O(1) + O(|V| + |E|) + O(|V|)$$

$$= O(|V| + |E|)$$



- 0 - Algorithms
- 1 - Complexity Theory
- 2 - AI
- 3 - Intro to CS
- 4 - Cryptograph
- 5 - Scientific Computing
- 6 - Advance programming

$$R = \{2, 5, 4, 1, 0, 6, 3\}$$

$$L = \{3, 6, 0, 1, 4, 5, 2\}$$

